

✓ AI활용프로그래밍 Week 9 실습 노트북

파일 입출력(File I/O) + 실습 데이터로 연습 (With AI)

본 노트북은 Week 9 강의자료를 바탕으로, **파일을 읽고/쓰는 기본 흐름**을 연습하도록 구성했습니다. [filecite](#) [turn4file0](#)

오늘 학습 목표

1. 텍스트 파일을 읽는 3가지 방법(read/readline/readlines)을 사용할 수 있다.
2. CSV/JSON 파일을 읽고 간단한 처리(합계, 평균, 빈도, TOP-N)를 할 수 있다.
3. 경로/작업 폴더(working directory) 개념을 이해한다.
4. LLM을 활용해 “코드 생성→검증→개선” 루틴을 적용한다.

실행 팁: 각 셀을 클릭하고 **Shift + Enter** 로 실행하세요.

✓ 0. 실습 데이터 준비 (Colab / Jupyter 공통)

이 노트북은 `week09_datafiles.zip` 데이터 묶음을 사용합니다.

- **Colab:** 왼쪽 파일 아이콘() → **업로드**로 zip 파일을 올린 뒤, 아래 셀을 실행하세요.
- **로컬 Jupyter:** 이 노트북(.ipynb)과 같은 폴더에 zip을 두고 아래 셀을 실행하면 됩니다.

✅ zip 파일이 없다면: 수업에서 제공된 `week09_datafiles.zip`을 사용하세요.

```
# (1) 현재 작업 폴더 확인
import os, pathlib
print("현재 작업 폴더:", os.getcwd())
print("폴더 내용 일부:", os.listdir('.')[0:10])
```

✓ 0-1. zip 파일 압축 해제

```
# (2) zip 파일을 풀어서 week09_datafiles/ 폴더를 만듭니다.
import zipfile

zip_name = "week09_datafiles.zip" # 필요하면 파일명을 바꾸세요.
extract_dir = "week09_datafiles"

if not os.path.exists(zip_name):
    print("⚠️ zip 파일이 현재 폴더에 없습니다:", zip_name)
    print("Colab이라면 왼쪽 📁에서 업로드 후 다시 실행하세요.")
else:
    with zipfile.ZipFile(zip_name, 'r') as z:
        z.extractall(".")
    print("✅ 압축 해제 완료!")
    print("폴더 확인:", os.listdir(extract_dir)[0:10])
```

✓ 1. 텍스트 파일 읽기: read / readline / readlines

✓ 1-1. read(): 파일 전체를 한 번에 읽기

```
path = "week09_datafiles/data.txt"

with open(path, "r", encoding="utf-8") as f:
    content = f.read()

print(content)
```

✓ 1-2. readline(): 한 줄씩 읽기(첫 줄만 예시)

```
with open(path, "r", encoding="utf-8") as f:
    line1 = f.readline()
```

```

line2 = f.readline()

print("1줄:", line1.strip())
print("2줄:", line2.strip())

```

✓ 1-3. readlines(): 줄 단위 리스트로 읽기

```

with open(path, "r", encoding="utf-8") as f:
    lines = f.readlines()

print("총 줄 수:", len(lines))
print("마지막 줄:", lines[-1].strip())

```

✓ 2. 숫자 파일 읽기: 공백/줄바꿈 섞인 데이터 처리

✓ 2-1. nums.txt를 읽어서 정수 리스트로 만들기

```

path_nums = "week09_datafiles/nums.txt"

with open(path_nums, "r", encoding="utf-8") as f:
    raw = f.read()          # 전체 문자열
    tokens = raw.split()    # 공백(스페이스/줄바꿈/탭) 기준 분리
    numbers = [int(x) for x in tokens]

print("원본 토큰 개수:", len(tokens))
print("숫자 리스트:", numbers)

```

✓ 2-2. 합계/평균/최대/최소 계산

```

total = sum(numbers)
avg = total / len(numbers)
print("합계:", total)
print("평균:", avg)
print("최대:", max(numbers))
print("최소:", min(numbers))

```

✓ 2-3. 엣지 케이스: 빈 파일(nums_empty.txt) 처리

빈 파일을 읽으면 숫자 리스트가 비어있습니다.

이때 평균을 구하면 **0으로 나누기 오류**가 날 수 있으니, 먼저 길이를 확인해야 합니다.

```

path_empty = "week09_datafiles/nums_empty.txt"

with open(path_empty, "r", encoding="utf-8") as f:
    raw_empty = f.read().strip()

tokens_empty = raw_empty.split() if raw_empty else []
numbers_empty = [int(x) for x in tokens_empty]

print("빈 파일 토큰:", tokens_empty)
print("빈 파일 숫자:", numbers_empty)

if len(numbers_empty) == 0:
    print("✅ 숫자가 없습니다. 평균 계산을 건너뛰니다.")
else:
    print("평균:", sum(numbers_empty) / len(numbers_empty))

```

✓ 3. CSV 읽기: grade.csv 성적 데이터

✓ 3-1. csv 모듈로 읽기

```
import csv

path_csv = "week09_datafiles/grade.csv"

rows = []
with open(path_csv, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        rows.append(row)

print("행 개수:", len(rows))
print("첫 행:", rows[0])
```

3-2. 각 학생의 평균 구하기

```
def to_int(x):
    return int(x)

for row in rows:
    name = row["name"]
    korean = to_int(row["korean"])
    math = to_int(row["math"])
    english = to_int(row["english"])
    avg = (korean + math + english) / 3
    print(f"{name}: avg={avg:.2f}")
```

3-3. 평균 TOP 3 구하기 (정렬)

```
avgs = []
for row in rows:
    name = row["name"]
    scores = [int(row["korean"]), int(row["math"]), int(row["english"])]
    avgs.append((name, sum(scores)/len(scores)))

# 평균 기준 내림차순 정렬
avgs_sorted = sorted(avgs, key=lambda x: x[1], reverse=True)

print("TOP 3")
for name, a in avgs_sorted[:3]:
    print(f"- {name}: {a:.2f}")
```

3-4. 결측치가 있는 CSV(grade_missing.csv) 다루기(심화)

grade_missing.csv에는 빈 칸(결측)이 있습니다.

오늘은 간단히 **빈 칸은 0점으로 처리**하는 방식으로 연습해봅니다.

```
path_missing = "week09_datafiles/grade_missing.csv"

def safe_int(x, default=0):
    x = (x or "").strip()
    return int(x) if x != "" else default

rows2 = []
with open(path_missing, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        rows2.append(row)

for row in rows2:
    name = row["name"]
    scores = [safe_int(row["korean"]), safe_int(row["math"]), safe_int(row["english"])]
    avg = sum(scores)/len(scores)
    print(f"{name}: scores={scores} avg={avg:.2f}")
```

4. JSON 읽기: settings.json / contacts_seed.json

4-1. settings.json 읽기

```
import json

path_settings = "week09_datafiles/settings.json"
with open(path_settings, "r", encoding="utf-8") as f:
    settings = json.load(f)

print(settings)
print("sound:", settings["sound"])
print("level:", settings["level"])
```

4-2. contacts_seed.json을 dict로 읽고 검색하기

```
path_contacts = "week09_datafiles/contacts_seed.json"
with open(path_contacts, "r", encoding="utf-8") as f:
    contacts = json.load(f)

print("초기 연락처:", contacts)

name = "Kim"
print(f"{name} 전화번호:", contacts.get(name, "(없음)"))
```

4-3. 연락처 추가/수정 후 다시 JSON으로 저장하기

```
contacts["Choi"] = "010-0000-0000" # 추가(또는 수정)
out_path = "week09_datafiles/contacts_updated.json"

with open(out_path, "w", encoding="utf-8") as f:
    json.dump(contacts, f, ensure_ascii=False, indent=2)

print("✅ 저장 완료:", out_path)
print("저장된 내용 일부:")
with open(out_path, "r", encoding="utf-8") as f:
    print(f.read())
```

5. 텍스트 분석 실습: 단어 빈도 TOP-N

5-1. sentence.txt 읽고 단어 빈도수 세기

```
path_sentence = "week09_datafiles/sentence.txt"

with open(path_sentence, "r", encoding="utf-8") as f:
    text = f.read().lower()

words = text.split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1

print("빈도:", freq)
```

5-2. TOP 3 출력

```
top3 = sorted(freq.items(), key=lambda x: x[1], reverse=True)[:3]
for w, c in top3:
    print(f"{w}: {c}")
```

6. set 실습: 로또 당첨 번호와 교집합

▼

6-1. lotto_winning.json 읽기

```
path_lotto = "week09_datafiles/lotto_winning.json"
with open(path_lotto, "r", encoding="utf-8") as f:
    lotto = json.load(f)

winning = set(lotto["winning_numbers"])
print("당첨 번호:", winning)
```

6-2. 내 번호(예시)와 교집합

```
my_numbers = {3, 7, 11, 20, 35, 45}
matched = my_numbers & winning

print("내 번호:", my_numbers)
print("맞춘 번호:", matched)
print("맞춘 개수:", len(matched))
```

7. 파일 쓰기 실습: 결과를 report.txt로 저장

```
report_path = "week09_datafiles/report.txt"

lines = []
lines.append("Week 9 실습 리포트")
lines.append("="*20)
lines.append(f"nums.txt 평균: {sum(numbers)/len(numbers):.2f}")
lines.append(f"grade.csv TOP1: {avgs_sorted[0][0]} ({avgs_sorted[0][1]:.2f})")
lines.append(f"word TOP1: {top3[0][0]} ({top3[0][1]})")
lines.append(f"lotto match count: {len(matched)}")

with open(report_path, "w", encoding="utf-8") as f:
    f.write("\n".join(lines))

print("✅ 저장 완료:", report_path)
print("—— 파일 내용 ——")
with open(report_path, "r", encoding="utf-8") as f:
    print(f.read())
```

8. With AI 프롬프트(추천)

8-1. 파일/경로 오류 질문 템플릿

아래 형식으로 AI에게 질문하면 정확도가 올라갑니다.

역할: 너는 파이썬 디버깅 코치야.
목표: 아래 오류를 해결하는 원인 후보 3개와 단계별 해결 절차를 알려줘.
내 환경:
- OS:
- 실행한 코드/명령어:
- 파일 위치(폴더 구조):
오류 메시지(그대로):

```
### 8-2. 코드 개선 요청 템플릿
```text
역할: 너는 파이썬 튜터야.
목표: 아래 코드를 더 안전하게(엣지 케이스 포함) 리팩터링해줘.
제약: 초보가 이해할 수 있게, 주석은 핵심만, 테스트 3개 포함.
코드:
```python
# 여기에 내 코드
```

9. 미니 퀴즈(개념 확인)

1. `open(..., encoding="utf-8")`를 쓰는 이유는?
2. `read()` vs `readlines()` 차이 한 가지는?
3. CSV에서 헤더(컬럼명)를 이용해 읽을 때 쓰는 도구는? (`csv.???`)
4. JSON 파일을 파이썬 dict로 읽는 함수는?
5. 파일이 비어있을 때 평균 계산에서 조심할 점은?

✅ 여기에 답을 적으세요:

- 1.
- 2.
- 3.
- 4.
- 5.

10. 과제(제출용)

기본 과제

1. `nums.txt`를 읽어 **합/평균/최대/최소**를 출력하고, 결과를 `result_nums.txt`로 저장
2. `sentence.txt` 단어 빈도 TOP 5를 출력하고, 결과를 `result_words.txt`로 저장
3. 실행 캡처 1장(터미널/노트북 출력이 보이게)

도전 과제

- `grade_missing.csv` 결측 처리 규칙을 **직접 정해서** 평균 TOP 3 출력
 - 예: 결측은 제외하고 평균 계산 / 결측은 0으로 처리 등
- 규칙을 README에 3줄로 설명

✅ 제출물: `.py` 1개(또는 `.ipynb`) + 결과 txt 2개 + 캡처 1장